ROUTIFY: OTIMIZAÇÃO DE ROTAS LOGÍSTICAS EM BRASÍLIA COM ALGORITMO A* E PREDIÇÃO DE TRÁFEGO VIA APRENDIZADO DE MÁQUINA

Pedro Borges Alves[1], Gustavo Martins Gripaldi[2] e Marcelo Alves Farias[3]

RESUMO: Este artigo testa a hipótese de que um modelo de aprendizado de máquina treinado sobre séries históricas de tráfego produz estimativas de tempo de viagem mais realistas do que o roteamento estático por menor distância. Para testá-la com tráfego real, foi construído o Routify, composto por um *pipeline* de coleta, um modelo XGBoost chamado LIA e um motor de roteamento A*. As cinco versões da LIA corrigiram defeitos como vazamento temporal na validação, *features* não reproduzíveis em produção e uma transferência de conhecimento sem calibração. Em validação cruzada temporal, a LIA 2.3 reduz o RMSE (raiz do erro quadrático médio) em 37,9% sobre o *baseline* histórico, de 57,2 para 35,5 segundos por trecho, e degrada pouco quando a leitura mais recente da via chega atrasada, ao contrário de uma referência de persistência. Por trecho, o RMSE é 66,4% menor que o de um preditor estático de fluxo livre. Por rota, em 2.289 viagens comparadas, a LIA erra menos que um roteador sem trânsito no pico de dia útil (erro percentual de 21,0% contra 26,1%, com a TomTom como referência), mas empata ou perde fora do pico. A validação externa preliminar, com a versão 2.1, mostra concordância direcional no atraso por congestionamento ($r=0,57$; $r^2=0,32$; $n=33$), com cerca de um terço da magnitude da referência. Um *benchmark* controlado descartou a migração para redes LSTM: o ganho aparente vinha do acesso à observação mais recente da via, vantagem replicável com o XGBoost. A hipótese é sustentada quando há congestionamento, não em trânsito leve.

Palavras-chave: Predição de tráfego; Aprendizado de máquina; Validação experimental; Algoritmo A*; XGBoost.

1 INTRODUÇÃO

O crescimento urbano e o consequente adensamento populacional representam desafios crescentes para a eficiência da mobilidade e da logística. Em Brasília, cuja malha viária tem características singulares de planejamento e fluxos concentrados, os atrasos gerados por congestionamentos impactam diretamente os custos operacionais de frotas e a agilidade das entregas. Parte do planejamento logístico urbano ainda se baseia em sistemas de roteamento estáticos, que calculam os trajetos pela menor distância física e ignoram a volatilidade do trânsito.

Soluções de navegação dinâmica de uso geral reduzem esse problema ao reagir às condições capturadas no momento da requisição e, nos sistemas mais avançados, ao combiná-las com padrões históricos em modelos proprietários (DERROW-PINION et al., 2021). Esses modelos, porém, não podem ser auditados nem ajustados ao contexto local. Além disso, o deslocamento até determinado trecho da rota pode durar dezenas de minutos, período em que as condições da via podem mudar. Esse cenário justifica investigar uma ferramenta preditiva e aberta à avaliação, alinhada às Cidades Inteligentes (*Smart Cities*).

Na interseção entre engenharia de software, modelagem de dados e inteligência artificial, áreas da Engenharia da Computação, o sistema proposto, batizado de Routify, integra o

[1] Graduando em Engenharia da Computação pelo Centro Universitário Instituto de Educação Superior de Brasília (IESB). E-mail: pedro.b.alves@iesb.edu.br
[2] Graduando em Engenharia da Computação pelo Centro Universitário Instituto de Educação Superior de Brasília (IESB). E-mail: gustavo.gripaldi@iesb.edu.br
[3] Orientador. Especialista em Governança de Tecnologia da Informação pela Universidade Católica de Brasília. E-mail: marcelo.a.farias@iesb.edu.br

algoritmo de busca em grafos A* a um modelo preditivo de aprendizado de máquina, apelidado de LIA (Logística de Inteligência Artificial), que ajusta os pesos (custo em tempo) das arestas (vias) do grafo com base no tráfego previsto para cada trecho.

O objetivo principal é testar, com tráfego real, a hipótese de que um modelo de aprendizado de máquina treinado sobre séries históricas de tráfego produz estimativas de tempo de viagem mais realistas do que o roteamento estático por distância, e investigar sob que condições essa vantagem se sustenta ou falha.

Os objetivos específicos são: (a) construir e validar um *pipeline* de coleta e preparação de dados de tráfego livre de vazamento temporal; (b) treinar e calibrar estatisticamente um modelo preditivo, corrigindo os defeitos metodológicos encontrados; (c) desenhar e executar um protocolo de validação externa que compare a previsão do modelo com uma referência comercial, sob as mesmas condições reais; e (d) construir a infraestrutura mínima (API, grafo de roteamento e aplicativo cliente) para que essa validação ocorra em operação real, não apenas em ambiente controlado de treino.

2 REVISÃO DE LITERATURA

2.1 Sistemas de Transporte Inteligentes e o Modelo Reativo vs. Proativo

Os Sistemas Inteligentes de Transporte (ITS, do inglês *Intelligent Transportation Systems*) representam a convergência entre tecnologias da informação, comunicação e infraestrutura viária (FIGUEIREDO et al., 2001). Na otimização de rotas urbanas, distinguem-se modelos reativos, que decidem com base em dados coletados em tempo real, e proativos, que estimam antecipadamente a saturação de trechos da malha a partir de tendências históricas e atributos de contexto. Plataformas comerciais como Google Maps e Waze apoiam-se fortemente em dados em tempo real obtidos por colaboração coletiva (*crowdsourcing*), e o Google Maps já combina esses dados com padrões históricos em redes neurais em grafos (DERROW-PINION et al., 2021). Como esses modelos são fechados, não é possível avaliar de forma independente seu erro por via nem adaptá-los ao planejamento de frotas, lacuna que motiva a avaliação aberta proposta aqui.

2.2 Teoria dos Grafos e o Algoritmo A*

A representação computacional de malhas viárias como redes é frequentemente tratada pela Teoria dos Grafos. Projetos como o OpenStreetMap (OSM) fornecem bases ricas em topologia, permitindo extrair nós (cruzamentos) e arestas (vias) tratáveis matematicamente (BOEING, 2017). Para determinar o caminho de menor custo, o algoritmo de Dijkstra (1959) expande-se de forma isotrópica, o que gera alto custo computacional em grafos da magnitude de cidades inteiras. O algoritmo A* (HART; NILSSON; RAPHAEL, 1968) reduz esse custo ao introduzir uma função heurística admissível, com $f(n) = g(n) + h(n)$, em que $g(n)$ é o custo acumulado e $h(n)$ estima o custo restante. No Routify, $g(n)$ é o tempo de viagem inferido pelo modelo preditivo, em vez da distância, e $h(n)$ é a distância Haversine até o destino dividida pela maior velocidade da malha, o que mantém a heurística em unidade de tempo e admissível.

2.3 Aprendizado de Máquina para Séries Temporais Logísticas

A inferência do tempo de viagem é modelada como um problema de regressão em séries temporais multivariadas. Abordagens clássicas como ARIMA e SARIMA têm dificuldade com os padrões não lineares e abruptos do fluxo urbano (MA et al., 2015). Modelos de *Gradient Boosting*, evoluídos a partir de Friedman (2001) e consagrados pelo XGBoost (CHEN; GUESTRIN, 2016), lidam bem com dados tabulares complexos, com alta velocidade de treinamento e tolerância a dados faltantes.

O XGBoost foi escolhido para o Routify pela facilidade de implantação e pela interpretabilidade estrutural. Parte da literatura, porém, indica desempenho superior de redes neurais LSTM (HOCHREITER; SCHMIDHUBER, 1997) na previsão de velocidade de tráfego (MA et al., 2015), dado seu controle explícito de esquecimento de estados da memória. Por isso, a Seção 6 avalia se essa vantagem se confirma no tráfego urbano de Brasília, em um *benchmark* controlado entre XGBoost e LSTM sob as mesmas condições de dados.

3 PROPOSTA E ARQUITETURA DO SISTEMA

Testar a hipótese central deste trabalho com tráfego real exige uma infraestrutura capaz de ingerir tráfego ao vivo, servir inferência em baixa latência e expor essa previsão a um cliente real. As escolhas a seguir, esquematizadas na Figura 1, atendem a esse requisito experimental.

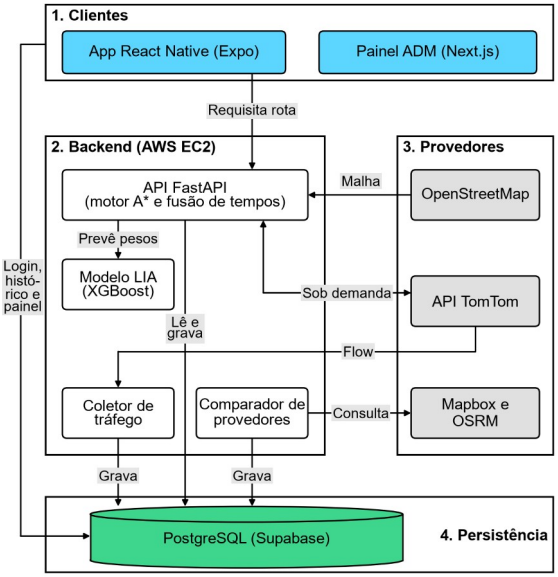
O cliente foi construído em React Native (Expo), para dispositivos móveis e web, o que permite operar o sistema de ponta a ponta e distingue este trabalho de uma avaliação puramente offline do modelo. Ele se comunica com uma API em FastAPI, que mantém o modelo preditivo e o grafo viário em memória para que a inferência ocorra na latência de uma única requisição, usando a leitura mais recente disponível, cuja idade é avaliada na Seção 6.

Os dados de tráfego (Seção 5.3) ficam centralizados em um banco Supabase (PostgreSQL); o histórico de março a julho de 2026 é a base de treino do modelo.

A rota é calculada pelo motor A* com os pesos de tempo de viagem previstos pela LIA (Seção 4.4). A API da TomTom é consultada sob demanda, no corredor dessa rota, para complementar a leitura de tráfego ao vivo e medir o tempo do mesmo trajeto, servindo de referência externa na validação experimental (Seção 6).

Um painel administrativo em Next.js, com permissão controlada no banco de dados e não no cliente, funciona como camada de observabilidade: mostra a qualidade dos dados e as métricas da LIA por versão e permite pausar a consulta à API da TomTom sem alterar código.

Figura 1 - Arquitetura em nuvem e fluxo de requisições do sistema Routify



Fonte: Elaborado pelos autores (2026).

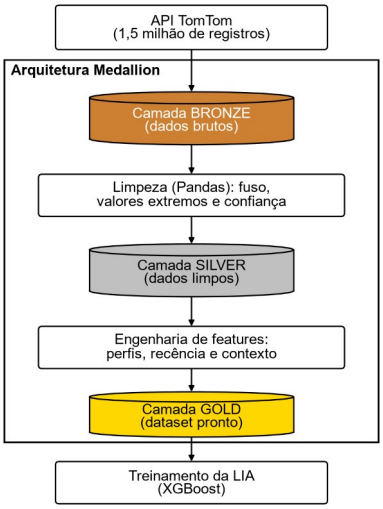
4 METODOLOGIA DE IMPLEMENTAÇÃO

O tratamento dos dados adota a arquitetura Medallion, padrão de engenharia de dados em camadas bronze, silver e gold descrito pela Databricks (DATABRICKS, 2026). A estrutura retoma a separação entre área de preparação e área de apresentação da modelagem dimensional (KIMBALL, 1996), adaptada a dados semiestruturados coletados periodicamente. A camada bronze contém o dado bruto, preservado na forma original para reprocessamento e auditoria; a silver, o dado validado, limpo e deduplicado; e a gold, o dado refinado e agregado, pronto para o treinamento do modelo (Figura 2).

A malha viária é representada como um grafo pela biblioteca OSMnx (BOEING, 2017), a partir de dados do OpenStreetMap, com nós

correspondendo a cruzamentos e arestas a trechos de via. O grafo é uma caixa de 76 × 76 km com cerca de 98 mil nós, que abrange 92,2% das vias dirigíveis do Distrito Federal.

Figura 2 - Funil de processamento e engenharia de dados sob a arquitetura Medallion



Fonte: Elaborado pelos autores (2026).

4.1 Camadas Bronze e Silver: Coleta, Limpeza e Engenharia de *Features*

A camada bronze é alimentada por um serviço que consulta ciclicamente a TomTom Flow Segment Data API, capturando velocidade atual, velocidade em fluxo livre, tempo de viagem estimado e grau de confiança para 630 pontos distribuídos pela malha arterial da cidade. Em novembro de 2026, a malha passou a 862 pontos, com 232 novos em regiões sem sensor, que ainda não entraram no treino.

Na camada silver, scripts em Python com a biblioteca Pandas padronizam o fuso horário das capturas, de UTC para o horário de Brasília, e descartam registros com velocidade fisicamente inviável ou confiança abaixo de um limiar mínimo. Lacunas pontuais de coleta recebem preenchimento por propagação, limitado a uma janela curta; lacunas maiores permanecem como ausência real de dado.

A engenharia de *features* combina atributos de calendário, perfil histórico e recência com a velocidade em fluxo livre da via, como mostra a seleção da Tabela 1.

Tabela 1 - Seleção das 21 *features* usadas no treinamento

Feature	Descrição
hora_sin / hora_cos	Hora do dia, em seno e cosseno
is_horario_pico	Indica horário de pico
velocidade_livre	Velocidade em fluxo livre da via
perfil_via_hora_dow	Mediana histórica por via, hora e dia da semana
razao_lag1	Última razão observada na via
delta_min_lag1	Tempo desde a última observação (log de minutos)
vizinhos_razao	Congestionamento médio das vias vizinhas
chuva_3h_mm	Chuva acumulada nas últimas 3 horas
is_feriado	Indica feriado nacional ou distrital

Fonte: Elaborado pelos autores (2026).

A LIA 2.2 acrescenta cinco *features* de contexto: congestionamento médio e número de vias vizinhas monitoradas, em raio de 3 km e janela de 60 minutos; chuva da última hora e das últimas três horas; e indicador de feriado, nacional ou distrital. O resultado é a camada gold, o *dataset* pronto para o treinamento (Seção 4.3).

4.2 Alvo, Perfis Históricos e Transferência de Conhecimento

O alvo de predição é a razão de congestionamento: velocidade atual dividida pela velocidade em fluxo livre do trecho, limitada ao intervalo de 0,05 a 1,0. Essa escolha torna o alvo independente do comprimento da via, permitindo que o mesmo modelo generalize entre trechos curtos e longos sem viés de escala.

Para cada via monitorada, constroem-se perfis históricos por via, hora do dia e dia da semana, usando a mediana da razão observada. Quando uma combinação não tem histórico suficiente, o sistema recorre a uma cascata de generalização progressiva, até a mediana global.

Uma *feature* de recência registra a última razão observada na via e o tempo decorrido

desde essa observação. Para vias sem monitoramento direto, uma técnica de transferência de conhecimento (*Knowledge Transfer*) localiza a via monitorada mais próxima, em um raio de até 500 m, e projeta seu perfil histórico para o segmento não monitorado.

Essa transferência foi calibrada por regressão isotônica não crescente (ZADROZNY; ELKAN, 2002), sobre a comparação entre perfis de pares de vias monitoradas em diferentes distâncias. A confiança decai com a distância, ponderando a predição transferida em direção ao fluxo livre.

4.3 Treinamento e Validação Cruzada Temporal

O modelo preditivo é um XGBoost Regressor (CHEN; GUESTRIN, 2016). Como os dados dependem do tempo, a validação usa TimeSeriesSplit (PEDREGOSA et al., 2011) com cinco *folds*, que respeita a ordem cronológica dos eventos e evita que informação futura vaze para o treinamento. Os perfis históricos são recalculados dentro de cada partição de treino, pelo mesmo motivo.

Duas referências são medidas em paralelo: um *baseline* que prediz diretamente o perfil histórico da via, sem aprendizado supervisionado, e uma referência de persistência, que repete a última razão observada na via, sem modelo algum.

A versão mais recente, a LIA 2.3, corrige três problemas de método: os perfis históricos do treino passam a ser calculados fora da amostra, em vez de incluir o próprio alvo da linha; metade dos exemplos de treino recebe uma recência artificialmente envelhecida, entre 15 e 480 minutos, para refletir o atraso real da leitura em produção; e a parada antecipada passa a usar o fim da própria partição de treino, não a validação que depois é medida.

4.4 Roteamento Dinâmico via A*

A integração ocorre em tempo de requisição. Dado um par origem-destino, o algoritmo A* busca o caminho de menor custo sobre o grafo viário. O peso de cada aresta é o tempo de viagem inferido a partir da razão de congestionamento prevista para aquele trecho, horário e dia da semana, com a heurística de tempo descrita na Seção 2.2.

Em seguida, a API da TomTom reconstrói esse trajeto e informa o tempo de viagem ao vivo, o que permite comparar as duas estimativas na mesma rota. O tempo exibido ao usuário combina as duas fontes, proporcionalmente à cobertura da LIA no trajeto. Origens e destinos a mais de 600 m da malha e alternativas claramente mais rápidas sugeridas pela TomTom são tratados pelo aplicativo e ficam fora desta avaliação, que considera apenas a rota da LIA.

5 DESAFIOS ARQUITETURAIS E SOLUÇÕES ADOTADAS

5.1 Grafo Não Navegável

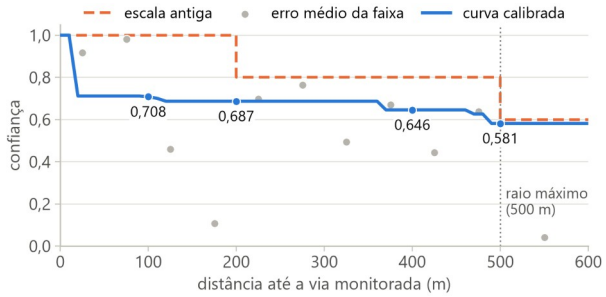
O grafo extraído do OpenStreetMap incluía arestas não motorizáveis, como passagens de pedestres e calçadas, o que gerava rotas inválidas, com o roteador cruzando áreas sem acesso veicular. A correção filtra essas arestas pela propriedade highway do OSM e remove vias de acesso proibido (access=no) na carga do grafo. Origem e destino são projetados na via mais próxima do maior componente fortemente conexo, no sentido permitido, e as restrições de conversão do OSM são respeitadas: em 300 viagens entre pontos de interesse do DF, as rotas com conversão proibida caíram de 70 para 0 e o trecho sem via até o ponto, de 164 para 124 m (mediana).

5.2 Vias Não Monitoradas e *Knowledge Transfer*

A coleta da TomTom cobre só vias arteriais monitoradas: no Distrito Federal, apenas 1,4% dos quilômetros de via ficam a menos de 50 m de um sensor. Sem cobertura direta, o roteador recorria a um peso heurístico estático, baseado só no limite de velocidade da via, descartando qualquer informação de congestionamento.

A transferência de conhecimento (Seção 4.2) reduz essa lacuna ao projetar o perfil da via monitorada mais próxima, alcançando mais 15,4% dos quilômetros de via. Os 83,2% restantes seguem sem sensor a menos de 500 m e usam velocidade livre calibrada e uma estimativa de congestionamento derivada dos trechos monitorados. A calibração por regressão isotônica, feita sobre 10.528 pares de vias monitoradas, substituiu uma escala fixa em degraus por uma confiança que decai com a distância (Figura 3).

Figura 3 - Confiança do *Knowledge Transfer* em função da distância: escala antiga e curva calibrada



Fonte: Elaborado pelos autores (2026).

A escala antiga era otimista em toda a faixa usada pela API: transferia mais padrão de congestionamento do que os dados sustentam, especialmente perto de 100 m, onde o valor calibrado é 0,71 contra 1,0. A curva calibrada também tem uma limitação: foi ajustada sobre a confiança de cada par já recortada em [0, 1], o que a deixa acima da média das faixas entre 100 e 500 m (cerca de 0,53, contra 0,58 a 0,71 da curva).

5.3 Cobertura Temporal da Coleta

A coleta cobriu 134 dias, de 7 de março a 19 de julho de 2026 (1.513.824 registros), quando foi interrompida porque os créditos da AWS que a hospedavam acabaram. A cadência nominal era de 8 minutos, mas a cota gratuita da API só sustentou o dia inteiro em 13 desses dias, que concentram 68,5% das linhas. Nos demais, a cobertura cai para uma mediana de 4 ciclos por dia, concentrados logo após a virada do dia em UTC: as 21h de Brasília respondem sozinhas por 23,3% das linhas, e um dia de fim de semana tem, em média, só 21% do volume de um dia útil.

Ela foi retomada em novembro de 2026, com créditos de outra conta, na mesma instância EC2 que serve a API e com cadência dimensionada pela cota (cerca de 43 minutos na janela avaliada).

5.4 Otimização de Hiperparâmetros

A otimização bayesiana de hiperparâmetros com Optuna (AKIBA et al., 2019), com amostrador TPE e poda por mediana, não superou a configuração manual nas mesmas condições de validação. O resultado é negativo, mas informativo: os hiperparâmetros manuais foram mantidos em produção.

5.5 Resiliência a Perdas de Conectividade (*Offline-First*)

O padrão *Offline-First* proposto no TCC1 não foi implementado neste ciclo, pois o esforço foi direcionado à validação do núcleo preditivo: sem conexão, o aplicativo perde comunicação com a API e pode interromper a navegação.

6 RESULTADOS E DISCUSSÕES

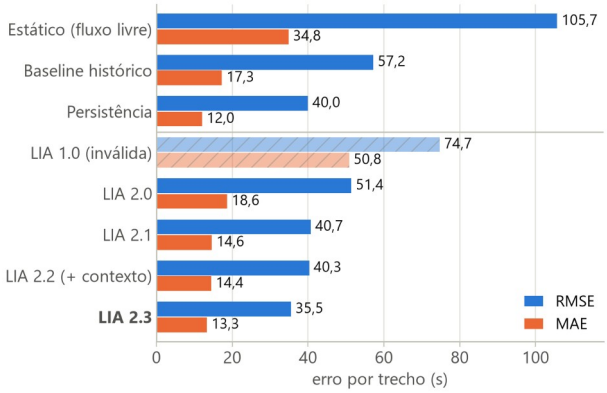
O conjunto de dados de treino da LIA 2.3 reúne 1.513.194 amostras limpas, e a validação segue o protocolo temporal de cinco *folds* (Seção 4.3). O erro é medido em segundos de tempo de viagem por trecho, convertendo a razão prevista r em tempo por $t = L / (v \cdot r)$, em que L é o comprimento do trecho e v, sua velocidade livre.

O MAE, erro médio absoluto, expressa o erro típico. O RMSE, ao elevar cada diferença ao quadrado antes da média, penaliza mais os erros grandes, concentrados nos trechos congestionados.

A Figura 4 resume as versões e as referências. O preditor estático assume fluxo livre em todo trecho (razão igual a 1), com a velocidade livre da TomTom, o que o favorece. A 1.0 usava validação que separava vias, não

períodos, e outro recorte de dados (667.104 amostras), por isso não é comparável. A 2.0 é o XGBoost sem a *feature* de recência, com números do *benchmark* descrito adiante; a 2.1 acrescenta a recência; a 2.2, o contexto de vizinhos, chuva e feriado; e a 2.3 corrige os três problemas de método da receita de treino. A persistência não é uma versão da LIA, e sim a referência sem modelo.

Figura 4 - Erro por versão da LIA e referências na validação cruzada temporal



Fonte: Elaborado pelos autores (2026).

A LIA 2.3 tem RMSE 66,4% menor que o do preditor estático (35,5 contra 105,7 s; MAE de 13,3 contra 34,8 s) e é melhor nos cinco *folds*.

Com leitura fresca, a persistência empata com a LIA 2.2 no RMSE e a supera no MAE (40,0 contra 40,3 s e 12,0 contra 14,4 s), porque nos dias de coleta densa a última leitura de uma via tem poucos minutos. Em produção, porém, a leitura chega com 0 a 48 minutos de idade, dado o ciclo de coleta ajustado pela cota.

Nessa condição, a persistência se degrada rapidamente: seu RMSE sobe de 40,0 s com leitura fresca para 64,8 s com 30 minutos de atraso e 88,0 s com 60 minutos. A LIA 2.3 degrada bem menos (35,5 s, 43,8 s e 47,2 s nos mesmos cenários), superando a persistência no RMSE em toda idade de leitura realista. A exceção é o erro típico com leitura muito recente, em que a persistência ainda vence no MAE (12,0 contra 13,3 s).

Em leituras que nenhum modelo viu (136.992 leituras de 2 a 8 de novembro de 2026, em todas as horas, com idade mediana de 46 minutos), a LIA 2.3 tem RMSE de 51,2 s, contra 61,6 s da persistência, 65,8 s da LIA 2.2 e 83,9 s do *baseline*. No MAE, porém, a persistência vence (19,4 contra 22,8 s), sobretudo de madrugada, quando o tráfego quase não muda. O erro maior que o da validação cruzada reflete, em parte, a cobertura uniforme dos picos nessa janela.

Essas leituras levaram à LIA 2.4, a mesma receita treinada também com os dados de novembro. Treinando só com o que existia antes de cada janela, ela reduziu o RMSE de 46,9 para 43,4 s e o MAE de 19,8 para 17,6 s em 77.772 leituras de 5 a 8 de novembro, vencendo a 2.3 nas duas janelas testadas, e foi adotada. Em 17.640 leituras de 9 de novembro, posteriores ao treino de todas as versões, a 2.4 teve o menor erro entre os modelos de produção (RMSE de 43,7 s e MAE de 19,5 s, contra 46,4 s e 22,1 s da 2.3 e 68,0 s e 26,3 s da persistência). Os demais resultados desta seção são da 2.3.

No subconjunto congestionado (razão abaixo de 0,95; 17,1% das amostras), o RMSE da LIA 2.3 é de 74,3 s, contra 257,1 s do estático, 123,7 s do *baseline* e 76,1 s da persistência. O erro dobra em relação ao conjunto total, e é aí que se concentra a limitação do modelo.

Um *benchmark* controlado testou a vantagem teórica do LSTM (Seção 2.3) sob as mesmas condições de dado: XGBoost sem recência (RMSE 51,4 s; MAE 18,6 s), LSTM (41,5 s; 17,1 s) e XGBoost com recência (41,0 s; 14,6 s, treinando cerca de três vezes mais rápido, sem exigir GPU). A vantagem do LSTM vinha do acesso à última observação da via, não da sazonalidade longa; com a mesma informação, o XGBoost empata no RMSE e vence no MAE, e a migração foi descartada pelo princípio da parcimônia.

A velocidade livre usada pela API vinha do limite de via do OpenStreetMap, enquanto o modelo foi treinado com a velocidade livre da TomTom. A diferença gerava um viés proporcional à distância, concentrado em vias

locais e coletoras, que levam cerca do dobro do tempo do limite de via. Fatores por grupo de via foram ajustados, junto com o atraso por semáforo (8,1 s), em 135 rotas comparadas à TomTom no mesmo trajeto: 0,909 em vias expressas, 0,897 em arteriais, 0,508 em coletoras e 0,425 em locais. Em validação fora da amostra (15 corredores, em um feriado, por volta de 12h), o viés mediano contra a TomTom foi de −9,1% para −4,0% e o erro percentual absoluto médio (MAPE), de 12,7% para 9,6%. Somando uma estimativa de congestionamento para arestas sem sensor, chegou a −2,3% e 8,7%. Como a referência é a própria TomTom, esses números medem aderência à escala dela, não acerto absoluto.

Dois outros testes não mostraram ganho: pesos por período do dia não corrigiram o viés no pico da tarde de dia útil (−11,3 s por trecho na LIA 2.3; −10,1 s no melhor caso), e sensores virtuais, leituras ao vivo da TomTom em pontos sem monitoramento, não reduziram o erro em dois testes em um feriado.

A comparação direta por rota usa o comparador automatizado em operação: 2.289 viagens em 86 rodadas (30 corredores, de 2 a 8 de novembro de 2026), cada uma com a LIA, o OSRM (LUXEN; VETTER, 2011), roteador sem trânsito que representa o roteamento estático, e as referências com trânsito TomTom e Mapbox. Contra a TomTom, no pico de dia útil (n=480), o MAPE da LIA é de 21,0%, contra 26,1% do OSRM (ganho de 4,3 a 5,9 pontos, IC de 95%); contra o Mapbox, que não participa do treino, é de 20,6% contra 25,4%. Fora do pico de dia útil (n=480), os dois empatam contra a TomTom (14,6% e 14,5%) e o OSRM vence contra o Mapbox (12,5% contra 15,1%). Em fins de semana e no feriado (n=1.329), o OSRM leva pequena vantagem (13,1% contra 13,8% e 10,5% contra 14,0%). A vantagem da LIA aparece, portanto, quando há congestionamento e desaparece em trânsito leve. No pico, ela ainda fica abaixo da referência (viés mediano de −20,4%, contra −26,8% do OSRM). TomTom e Mapbox divergem entre si em 9,4%, piso prático dessa medida.

A validação externa compara o atraso por congestionamento previsto (tempo no instante da consulta menos tempo em horário de referência sem tráfego) com o atraso informado pela API de rotas da TomTom, para os mesmos trajetos e instantes. A primeira rodada, feita com a LIA 2.1 em dois dias úteis (24 de agosto e 18 de setembro de 2026, às 7h, 9h e 13h), mostra concordância direcional preliminar ($r=0,57$; $r^2=0,32$; $n=33$), mas subestima a magnitude: a soma do atraso previsto foi de 642 s, contra 1.988 s da referência, cerca de um terço. As causas prováveis são o desbalanceamento dos dados (83% das amostras em fluxo livre), a perda quadrática, que puxa as previsões para a média, e a cobertura enviesada do treino (Seção 5.3).

A TomTom é uma referência externa, mas não independente, pois os dados de treino vêm da API de fluxo da mesma empresa; por isso o comparador registra também o Mapbox. Como checagem sem relação com a TomTom, a contagem de veículos do DER-DF (DISTRITO FEDERAL, 2026), em 93 trechos e 45 dias úteis, acompanha a lentidão do *dataset* por hora ($r=0,80$). A medição de atraso ainda não foi repetida com a LIA 2.3.

7 CONSIDERAÇÕES FINAIS

Este trabalho testou, com tráfego real e não apenas em bancada, a hipótese de que um modelo de aprendizado de máquina treinado sobre séries históricas produz estimativas de tempo de viagem mais realistas do que o roteamento estático. A evidência sustenta a hipótese quando há congestionamento. Por trecho, o RMSE da LIA 2.3 é 66,4% menor que o do preditor estático e 37,9% menor que o do *baseline* histórico, e o modelo mantém a vantagem sobre a persistência no RMSE quando a leitura chega atrasada, inclusive em leituras que nenhum modelo viu (51,2 contra 61,6 s), embora perca no MAE. Por rota, a LIA erra menos que o roteador sem trânsito no pico

de dia útil (21,0% contra 26,1%), mas a vantagem desaparece em trânsito leve, quando a velocidade livre já é uma boa estimativa. A previsão de atraso concorda em direção com a referência externa, mas fica em cerca de um terço de sua magnitude. Com mais dados, a LIA 2.4 superou a 2.3 em teste fora do tempo.

A hipótese de que redes LSTM superariam o XGBoost foi refutada, e a otimização bayesiana, os pesos por período e os sensores virtuais não trouxeram ganho consistente; já a calibração revelou que a escala inicial de transferência superestimava a confiança.

As principais limitações são: (a) a LIA ainda subestima o pico; a validação de atraso tem amostra pequena (33 pares em dois dias úteis) e usa a versão 2.1; o comparador de rotas cobre só uma semana, com um feriado; e a TomTom não é referência independente; (b) a cobertura temporal do treino é enviesada: só 13 dos 134 dias de coleta tiveram cobertura completa; e (c) a dependência de uma API comercial gratuita, sujeita a cotas e termos de uso, é um risco de disponibilidade, adequado apenas a um protótipo acadêmico.

O sistema permanece em operação, com API, aplicativo e painel administrativo de observabilidade. Como trabalhos futuros, destacam-se reduzir a subestimação no pico, repetir a validação de atraso com a versão atual, treinar a LIA com os 232 pontos adicionados nas regiões sem sensor e implementar o modo *Offline-First*.

ABSTRACT: *This paper tests the hypothesis that a machine-learning model trained on historical traffic series produces more realistic travel-time estimates than static, shortest-distance routing. To test it with real traffic, Routify was built: a data collection pipeline, an XGBoost model called LIA and an A* routing engine. LIA's five versions fixed methodological defects such as temporal leakage in validation, features not reproducible in production and uncalibrated knowledge transfer. In temporal cross-validation, LIA 2.3 reduces RMSE (root mean squared error) by 37.9% over the historical baseline, from 57.2 to 35.5 seconds per segment, and degrades little when the road's most recent reading arrives late, unlike a persistence reference. Per segment, RMSE is 66.4% lower than that of a static free-flow predictor. Per route, over 2,289 compared trips, LIA errs less than a traffic-free router in weekday peaks (percentage error of 21.0% vs. 26.1%, with TomTom as reference), but ties or loses off-peak. A preliminary external validation, run with version 2.1, shows directional agreement on congestion delay ($r=0.57$; $r^2=0.32$; $n=33$), at about one third of the reference's magnitude. A controlled benchmark discarded migrating to LSTM networks: the apparent gain came from access to the road's most recent observation, an advantage replicable with XGBoost. The hypothesis is supported under congestion, not under light traffic.*

Keywords: *Traffic prediction; Machine learning; Experimental validation; A* algorithm; XGBoost.*

REFERÊNCIAS

AKIBA, T. et al. Optuna: a next-generation hyperparameter optimization framework. In: ACM SIGKDD INTERNATIONAL CONFERENCE ON KNOWLEDGE DISCOVERY AND DATA MINING, 25., 2019, Anchorage. **Proceedings** [...]. New York: ACM, 2019. p. 2623-2631.

BOEING, G. OSMnx: new methods for acquiring, constructing, analyzing, and visualizing complex street networks. **Computers, Environment and Urban Systems**, v. 65, p. 126-139, 2017.

CHEN, T.; GUESTRIN, C. XGBoost: a scalable tree boosting system. In: ACM SIGKDD INTERNATIONAL CONFERENCE ON KNOWLEDGE DISCOVERY AND DATA MINING, 22., 2016, San Francisco. **Proceedings** [...]. New York: ACM, 2016. p. 785-794.

DATABRICKS. **What is the medallion lakehouse architecture?** [S. l.]: Databricks, 2026. Disponível em: https://docs.databricks.com/aws/en/lakehouse/medallion. Acesso em: 8 nov. 2026.

DERROW-PINION, A. et al. ETA prediction with graph neural networks in Google Maps. In: ACM INTERNATIONAL CONFERENCE ON INFORMATION AND KNOWLEDGE MANAGEMENT, 30., 2021, Queensland. **Proceedings** [...]. New York: ACM, 2021. p. 3767-3776.

DIJKSTRA, E. W. A note on two problems in connexion with graphs. **Numerische Mathematik**, v. 1, n. 1, p. 269-271, 1959.

DISTRITO FEDERAL. Departamento de Estradas de Rodagem. **Dados de fluxo veicular**. Brasília: DER-DF, 2026. Disponível em: https://dados.df.gov.br/o/dados-abertos/v1.0/datasets/volume-diario-de-trafego-trechos-rodoviarios. Acesso em: 9 nov. 2026.

FIGUEIREDO, L. et al. Towards the development of intelligent transportation systems. In: IEEE INTELLIGENT TRANSPORTATION SYSTEMS CONFERENCE, 4., 2001, Oakland. **Proceedings** [...]. Piscataway: IEEE, 2001. p. 1206-1211.

FRIEDMAN, J. H. Greedy function approximation: a gradient boosting machine. **Annals of Statistics**, v. 29, n. 5, p. 1189-1232, 2001.

HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A formal basis for the heuristic determination of minimum cost paths. **IEEE Transactions on Systems Science and Cybernetics**, v. 4, n. 2, p. 100-107, 1968.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. **Neural Computation**, v. 9, n. 8, p. 1735-1780, 1997.

KIMBALL, R. **The data warehouse toolkit**: practical techniques for building dimensional data warehouses. New York: John Wiley & Sons, 1996.

LUXEN, D.; VETTER, C. Real-time routing with OpenStreetMap data. In: ACM SIGSPATIAL INTERNATIONAL CONFERENCE ON ADVANCES IN GEOGRAPHIC INFORMATION SYSTEMS, 19., 2011, Chicago. **Proceedings** [...]. New York: ACM, 2011. p. 513-516.

MA, X. et al. Long short-term memory neural network for traffic speed prediction using remote microwave sensor data. **Transportation Research Part C: Emerging Technologies**, v. 54, p. 187-197, 2015.

PEDREGOSA, F. et al. Scikit-learn: machine learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825-2830, 2011.

ZADROZNY, B.; ELKAN, C. Transforming classifier scores into accurate multiclass probability estimates. In: ACM SIGKDD INTERNATIONAL CONFERENCE ON KNOWLEDGE DISCOVERY AND DATA MINING, 8., 2002, Edmonton. **Proceedings** [...]. New York: ACM, 2002. p. 694-699.

AGRADECIMENTOS

Agradecemos ao nosso orientador, Prof. Marcelo Alves Farias, pelo direcionamento contínuo durante o desenvolvimento deste trabalho. Estendemos os agradecimentos ao corpo docente do Centro Universitário Instituto de Educação Superior de Brasília (IESB) por todo o conhecimento provido durante o curso de Engenharia da Computação, bem como aos nossos familiares pelo apoio incondicional.